NAME: M. Loganath

REG NO.:192424399

COURSE CODE: CSA1748

COURSE NAME: Artificial Intelligence For Healthcare Systems

1. Write a Prolog Program to Sum the Integers from 1 to n.

sum.pl

```
sum(0,0).
sum(N,Sum) :-
 N > 0,
 N1 is N - 1,
 sum(N1,Sum1),
 Sum is Sum1 + N.
```

GNU Prolog console

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/sum.pl').
compiling C:/Users/logan/OneDrive/Documents/sum.pl for byte code...
C:/Users/logan/OneDrive/Documents/sum.pl compiled, 5 lines read - 824 bytes written

yes
| ?- sum(5,Sum).

Sum = 15 ?

yes
| ?-  sum(10,Sum).

Sum = 55 ?

yes
| ?-
```

2. Write a Prolog Program for A DB WITH NAME, DOB.

dob.pl

```
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).
dob(sue, date(1995, 8, 15)).
dob(tom, date(2000, 4, 22)).

lookup(Name, DOB) :-
dob(Name, DOB).
```

GNU Prolog console

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Desktop/dob.pl').
compiling C:/Users/logan/OneDrive/Desktop/dob.pl for byte code...
C:/Users/logan/OneDrive/Desktop/dob.pl compiled, 7 lines read - 1177 bytes written,

(15 ms) yes
| ?- lookup(Name,DOB).

DOB = date(1990,5,1)
Name = john ?

yes
| ?- lookup(bob,DOB).

DOB = date(1978,2,28)

yes
| ?-
```

3. Write a Prolog Program for STUDENT TEACHER-SUB-CODE.

teacher-student.pl

```
teaches(john, math).
teaches(jane, english).
teaches(bob, science).
teaches(sue, history).
teaches(tom, art).

takes(alice, math).
takes(alice, science).
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).


teaching_subjects(Teacher, Subject) :-
teaches(Teacher, Subject).


taking_students(Subject, Student) :-
takes(Student, Subject).
```

GNU Prolog console

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/teacher-student.pl').
compiling C:/Users/logan/OneDrive/Documents/teacher-student.pl for byte code...
C:/Users/logan/OneDrive/Documents/teacher-student.pl compiled, 23 lines read - 1912

yes
| ?- taking_students(science, Student).

Student = alice ?

(16 ms) yes
| ?- taking_students(math, Student), taking_students(art, Student).

Student = dave ?

yes
| ?-
```

4. Write a Prolog Program for PLANETS DB.

sum.pl | teacher-student.pl | dob.pl | plant.pl

File Edit View

```
planet(mercury, rocky, small, hot, closest_to_sun).
planet(venus, rocky, small, hot, '2nd_closest_to_sun').
planet(earth, rocky, medium, temperate, '3rd_closest_to_sun').
planet(mars, rocky, small, cold, '4th_closest_to_sun').
planet(jupiter, gas_giant, large, cold, '5th_closest_to_sun').
planet(saturn, gas_giant, large, cold, '6th_closest_to_sun').
planet(uranus, ice_giant, large, cold, '7th_closest_to_sun').
planet(neptune, ice_giant, large, cold, '8th_closest_to_sun').

planet_properties(Name, Type, Size, Temperature, Position) :-
    planet(Name, Type, Size, Temperature, Position).
```

GNU Prolog console

File Edit Terminal Prolog Help

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/plant.pl').
compiling C:/Users/logan/OneDrive/Documents/plant.pl for byte code...
C:/Users/logan/OneDrive/Documents/plant.pl compiled, 10 lines read - 2058 bytes wri

yes
| ?- planet_properties(earth, Type, Size, Temperature, Position).

Position = '3rd_closest_to_sun'
Size = medium
Temperature = temperate
Type = rocky

yes
| ?- planet_properties(Name, _, small, hot, _).

Name = mercury ?

yes
| ?-
```

5. Write a Prolog Program to implement Towers of Hanoi.

sum.pl | teacher-student | dob.pl | plant.pl | tower.pl

File Edit View

```
% Base case: move 1 disk directly
move(1, From, To, _) :-
    write('Move the top disk from '), write(From),
    write(' to '), write(To), nl.

% Recursive case: move N disks
move(N, From, To, Aux) :-
    N > 1,
    N1 is N - 1,
    move(N1, From, Aux, To),
    move(1, From, To, _),
    move(N1, Aux, To, From).
```

GNU Prolog console

File Edit Terminal Prolog Help

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/tower.pl').
compiling C:/Users/logan/OneDrive/Documents/tower.pl for byte code...
C:/Users/logan/OneDrive/Documents/tower.pl compiled, 11 lines read - 1417 bytes wri

yes
| ?- move(3, left, right, center).
Move the top disk from left to right
Move the top disk from left to center
Move the top disk from right to center
Move the top disk from left to right
Move the top disk from center to left
Move the top disk from center to right
Move the top disk from left to right

true ?
```

6. Write a Prolog Program to print particular bird can fly or not. Incorporate required queries.

sum.pl | teacher-stud | dob.pl | plant.pl | tower.pl | fly.pl

File Edit View

```
bird(ostrich, cannot_fly).
bird(penguin, cannot_fly).
bird(kiwi, cannot_fly).
bird(eagle, can_fly).
bird(pigeon, can_fly).
bird(sparrow, can_fly).

can_fly(Bird) :-
    bird(Bird, can_fly).
```

GNU Prolog console

File Edit Terminal Prolog Help

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/fly.pl').
compiling C:/Users/logan/OneDrive/Documents/fly.pl for byte code...
C:/Users/logan/OneDrive/Documents/fly.pl compiled, 8 lines read - 940 bytes written

(16 ms) yes
| ?- can_fly(eagle).

yes
| ?- cannot_fly(ostrich).
uncaught exception: error(existence_error(procedure,cannot_fly/1),top_level/0)
| ?- can_fly(B).

B = eagle ?

yes
| ?- can_fly(penguin).

no
| ?-
```

7. Write the prolog program to implement family tree Pam, Liz, Ann and Pat are female, while Tom, Bob and Jim are male persons. Using this information, define the following relations: • Define the “mother” relation: • Define the “father” relation: • Define the “grandfather” relation: • Define the “grandmother” relation: • Define the “sister” relation Define the “brother” relation

```
male(bob).
male(jim).

% Parent relations
parent(tom, liz).
parent(tom, bob).
parent(pam, liz).
parent(pam, bob).
parent(bob, ann).
parent(bob, pat).
parent(liz, jim).

% Mother relation
mother(X, Y) :-
    female(X),
    parent(X, Y).

% Father relation
father(X, Y) :-
    male(X),
    parent(X, Y).

% Grandmother relation
grandmother(X, Y) :-
    female(X),
    parent(X, Z),
    parent(Z, Y).

% Grandfather relation
grandfather(X, Y) :-
    male(X),
    parent(X, Z),
    parent(Z, Y).

% Sister relation
sister(X, Y) :-
    female(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

% Brother relation
brother(X, Y) :-
    male(X),
    parent(Z, X),
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/grandmother.pl').
compiling C:/Users/logan/OneDrive/Documents/grandmother.pl for byte code...
C:/Users/logan/OneDrive/Documents/grandmother.pl compiled, 53 lines read - 3749 by

(31 ms) yes
| ?- ?- mother(pam, liz).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.
| ?-
?- father(tom, bob).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- grandmother(pam, ann).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- brother(bob, liz).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- sister(liz, bob).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- grandfather(X, jim).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- X = tom.
```

8. Write a Prolog Program to suggest Dieting System based on Disease.

```
% Define the foods and their properties
food(apple, fruit, sweet, low_calorie).
food(banana, fruit, sweet, high_calorie).
food(carrot, vegetable, savory, low_calorie).
food(potato, vegetable, savory, high_calorie).
food(chicken, meat, savory, high_protein).
food(fish, seafood, savory, high_protein).
food(spinach, vegetable, savory, high_iron).
food(almonds, nut, savory, high_fat).

% Define the recommended diet based on the disease
diet(heart_disease, [apple, carrot, chicken, fish, almonds]).
diet(diabetes, [apple, carrot, fish, spinach, almonds]).
diet(anemia, [spinach, chicken, fish, almonds]).
diet(obesity, [apple, carrot, fish, spinach]).

% Define the rules to suggest the diet based on the disease
suggest_diet(Disease, Diet) :-
    diet(Disease, Diet).

suggest_diet(Disease, Diet) :-
    diet(Disease, AllowedFoods),
    findall(Food, (food(Food, _, _, _), member(Food, AllowedFoods)), Diet).
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- suggest_diet(diabetes, Diet).
uncaught exception: error(existence_error(procedure,suggest_diet/2),top_level/0)
| ?- consult('C:/Users/logan/OneDrive/Documents/food.pl').
compiling C:/Users/logan/OneDrive/Documents/food.pl for byte code...
C:/Users/logan/OneDrive/Documents/food.pl compiled, 22 lines read - 3455 bytes writ

yes
| ?- suggest_diet(diabetes, Diet).

Diet = [apple,carrot,fish,spinach,almonds] ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
Action (; for next solution, a for all solutions, RET to stop) ?
```

9. Write a Prolog program to implement Monkey Banana

```
% Initial and final states
initial_state(state(at_door, on_floor, at_window, has_not_eaten)).
final_state(state(_, _, _, has_eaten)).

% Actions
action(state(at_door, on_floor, at_window, has_not_eaten), climb, state(at_window,
on_window, at_window, has_not_eaten)).

action(state(at_window, on_floor, at_window, has_not_eaten), grasp, state(at_window,
on_floor, at_window, has_eaten)).

action(state(at_window, on_window, at_window, has_not_eaten), climb, state(at_door,
on_window, at_door, has_not_eaten)).

action(state(at_door, on_window, at_door, has_not_eaten), walk, state(at_middle,
on_floor, at_middle, has_not_eaten)).

action(state(at_middle, on_floor, at_middle, has_not_eaten), grasp, state(at_middle,
on_floor, at_middle, has_eaten)).

action(state(at_middle, on_floor, at_middle, has_eaten), walk, state(at_door, on_floor,
at_door, has_eaten)).

% Execute a sequence of actions
execute_actions(_, [],FinalState) :-final_state(CurrentState).

execute_actions(CurrentState, [Action|Rest], FinalState) :-
    action(CurrentState, Action, NextState),
    execute_actions(NextState, Rest, FinalState).

% Solve the problem
solve_problem(ActionList) :-
    initial_state(InitialState),
    execute_actions(InitialState, ActionList, FinalState),
    final_state(FinalState).
```

```
compiling C:/Users/logan/OneDrive/Documents/monkey banana.pl for byte code...
C:/Users/logan/OneDrive/Documents/monkey banana.pl:25: warning: singleton variables
C:/Users/logan/OneDrive/Documents/monkey banana.pl compiled, 35 lines read - 4357 b

(15 ms) yes
| ?- solve_problem(ActionList).

ActionList = [] ? a

ActionList = [climb]

ActionList = [climb,climb]

ActionList = [climb,climb,walk]

ActionList = [climb,climb,walk,grasp]

ActionList = [climb,climb,walk,grasp,walk]

(32 ms) no
| ?- ction(state(at_door, on_floor, at_window, has_not_eaten), climb, state(at_wind
on_window, at_window, has_not_eaten)).
uncaught exception: error(existence_error(procedure,ction/3),top_level/0)
| ?- action(state(at_window, on_floor, at_window, has_not_eaten), grasp, state(at_w
on_floor, at_window, has_eaten)).

true ?

yes
| ?- action(state(at_window, on_window, at_window, has_not_eaten), climb, state(at_
on_window, at_door, has_not_eaten)).

true ?

yes
| ?- action(state(at_door, on_window, at_door, has_not_eaten), walk, state(at_middl
on_floor, at_middle, has_not_eaten)).

true ?

yes
| ?- action(state(at_middle, on_floor, at_middle, has_not_eaten), grasp, state(at_m
on_floor, at_middle, has_eaten)).

true ?

yes
| ?- action(state(at_middle, on_floor, at_middle, has_eaten), walk, state(at_door,
at_door, has_eaten)).

yes
| ?- % Define a predicate to execut
```

10. Write a Prolog Program for fruit and its color using Back Tracking.

```
% Facts: fruit and their colors
fruit(apple, red).
fruit(banana, yellow).
fruit(grape, purple).
fruit(orange, orange).
fruit(watermelon, green).

% Rule: match a fruit with its color
match_fruit_color(Fruit, Color) :-
    fruit(Fruit, Color).

% Rule: find all fruits of a given color
fruits_with_color(FruitList, Color) :-
    findall(Fruit, match_fruit_color(Fruit, Color), FruitList).
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/fruit backtracking.pl').
compiling C:/Users/logan/OneDrive/Documents/fruit backtracking.pl for byte code...
C:/Users/logan/OneDrive/Documents/fruit backtracking.pl compiled, 13 lines read - 1

(16 ms) yes
| ?-  match_fruit_color(apple, Color).

Color = red

(15 ms) yes
| ?- match_fruit_color(banana, Color).

Color = yellow

yes
| ?- fruits_with_color(FruitList, green).

FruitList = [watermelon]

yes
| ?-
```

11. Write a Prolog Program to implement Best First Search algorithm

```
% ------------------------------------
% Graph representation (edges with cost)
% ------------------------------------
edge(a, b, 3).
edge(a, c, 6).
edge(a, d, 5).
edge(b, e, 9).
edge(b, f, 8).
edge(c, g, 12).
edge(c, h, 14).
edge(d, i, 7).
edge(i, j, 5).
edge(i, k, 6).
edge(j, l, 1).
edge(j, m, 10).
edge(j, n, 2).

% ------------------------------------
% Heuristic values (example)
% ------------------------------------
heuristic(a, 10).
heuristic(b, 8).
heuristic(c, 7).
heuristic(d, 6).
heuristic(e, 5).
heuristic(f, 5).
heuristic(g, 4).
heuristic(h, 4).
heuristic(i, 3).
heuristic(j, 2).
heuristic(k, 2).
heuristic(l, 0).   % Goal node
heuristic(m, 1).
heuristic(n, 1).

% ------------------------------------
% Best First Search Algorithm
% ------------------------------------
best_first_search(Start, Goal, Path) :-
    bfs([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

% BFS helper: expand paths based on heuristic
bfs([[Goal|Rest]|_], Goal, [Goal|Rest]).
bfs([Path|Paths], Goal, FinalPath) :-
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/bfs.pl').
compiling C:/Users/logan/OneDrive/Documents/bfs.pl for byte code...
C:/Users/logan/OneDrive/Documents/bfs.pl compiled, 63 lines read - 5964 bytes writt

(15 ms) yes
| ?- best_first_search(a, l, Path).
uncaught exception: error(existence_error(procedure,map_list_to_pairs/3),sort_paths
| ?- consult('C:/Users/logan/OneDrive/Documents/bfs.pl').
compiling C:/Users/logan/OneDrive/Documents/bfs.pl for byte code...
C:/Users/logan/OneDrive/Documents/bfs.pl compiled, 77 lines read - 6832 bytes writt

(16 ms) yes
| ?- best_first_search(a, l, Path).

Path = [a,d,i,j,l] ?
```

12. Write the prolog program for Medical Diagnosis

```
% ------------------------------------
% Facts: Symptoms associated with diseases
% ------------------------------------
symptom(fever, flu).
symptom(cough, flu).
symptom(body_ache, flu).
symptom(sore_throat, flu).

symptom(fever, malaria).
symptom(chills, malaria).
symptom(sweating, malaria).
symptom(headache, malaria).

symptom(chest_pain, pneumonia).
symptom(cough, pneumonia).
symptom(fever, pneumonia).
symptom(difficulty_breathing, pneumonia).

symptom(runny_nose, cold).
symptom(sneezing, cold).
symptom(cough, cold).
symptom(sore_throat, cold).

% ------------------------------------
% Rule: A disease is possible if all its symptoms are present
% ------------------------------------
has_disease(Disease, PatientSymptoms) :-
    findall(S, symptom(S, Disease), DiseaseSymptoms),
    subset(DiseaseSymptoms, PatientSymptoms).

% ------------------------------------
% Utility: Check if all elements of one list are in another
% ------------------------------------
subset([], _).
subset([H|T], List) :-
    member(H, List),
    subset(T, List).
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/health.pl').
compiling C:/Users/logan/OneDrive/Documents/health.pl for byte code...
C:/Users/logan/OneDrive/Documents/health.pl compiled, 28 lines read - 2684 bytes wr

yes
| ?- has_disease(flu, [fever, cough, body_ache, sore_throat]).

true ? h
Action (; for next solution, a for all solutions, RET to stop) ? a

no
| ?- s_disease(malaria, [fever, chills, headache]).
uncaught exception: error(existence_error(procedure,s_disease/2),top_level/0)
| ?- consult('C:/Users/logan/OneDrive/Documents/health.pl').
compiling C:/Users/logan/OneDrive/Documents/health.pl for byte code...
C:/Users/logan/OneDrive/Documents/health.pl compiled, 36 lines read - 2684 bytes wr

(15 ms) yes
| ?- ?- has_disease(flu, [fever, cough, body_ache, sore_throat]).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- has_disease(malaria, [fever, chills, headache]).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- false.   % because sweating is missing

no
| ?-
?- has_disease(cold, [sneezing, runny_nose, cough, sore_throat]).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- true.

yes
| ?-
?- has_disease(pneumonia, [fever, cough, chest_pain]).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- false.   % difficulty_breathing is missing

no
| ?- has_disease(flu, [fever, cough, body_ache, sore_throat]).

true ?
```

13. Write a Prolog Program for forward Chaining. Incorporate required queries.

```
% Define the facts and rules
bird(penguin) :- cannot_fly.
bird(eagle) :- can_fly.
cannot_fly.
can_fly :- has_wings.
has_wings.
% Define the forward chaining rule
infer(X) :- bird(X).
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/forward chaining.pl').
compiling C:/Users/logan/OneDrive/Documents/forward chaining.pl for byte code...
C:/Users/logan/OneDrive/Documents/forward chaining.pl compiled, 9 lines read - 1227

yes
| ?- infer(crow).

no
| ?- infer(X).

X = penguin ?

yes
| ?- infer(eagle).

true ?

(16 ms) yes
| ?-
```

14. Write a Prolog Program for backward Chaining. Incorporate required queries.

```
% Facts
fact(rain).
fact(cloudy).

% Rules
rule(wet_ground, [rain]).
rule(carry_umbrella, [cloudy]).
rule(stay_home, [wet_ground, carry_umbrella]).

% Backward chaining engine
prove(Goal) :-
    fact(Goal).

prove(Goal) :-
    rule(Goal, Conditions),
    prove_all(Conditions).

prove_all([]).
prove_all([H|T]) :-
    prove(H),
    prove_all(T).
```

```
GNU Prolog 1.5.0 (32 bits)
Compiled Jul  8 2021, 12:47:53 with gcc
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/logan/OneDrive/Documents/backward chaining.pl').
compiling C:/Users/logan/OneDrive/Documents/backward chaining.pl for byte code...
C:/Users/logan/OneDrive/Documents/backward chaining.pl compiled, 20 lines read - 15

yes
| ?- % Facts
fact(rain).

yes
| ?- fact(cloudy).

yes
| ?-
% Rules
rule(wet_ground, [rain]).

yes
| ?- rule(carry_umbrella, [cloudy]).

yes
| ?- rule(stay_home, [wet_ground, carry_umbrella]).

yes
| ?-
% Backward chaining engine
prove(Goal) :-
    fact(Goal).
uncaught exception: error(existence_error(procedure,(:-)/2),top_level/0)
| ?-
prove(Goal) :-
    rule(Goal, Conditions),
    prove_all(Conditions).
uncaught exception: error(existence_error(procedure,(:-)/2),top_level/0)
| ?-
prove_all([]).

yes
| ?- prove_all([H|T]) :-
    prove(H),
    prove_all(T).
```